



A Predictive Modelling Approach to Car Price Estimation

Group members: 250497683, 250454930, 250394647, 250357343, 250123069

250123069;Candidate No: 212515

Predictive Analytics BBM106

2/13/26

Project Summary

This project developed a predictive model to predict car prices for possible stakeholders like a Car Dealership company, Buyers/Sellers, Insurance/Financial Institutions, etc. The car price prediction (on a given dataset of ~2200 records) is based on key vehicle features such as mileage, fuel type, service history, ownership, safety, and transmission. Multiple models were tested, including Random Forest, XGBoost, Neural Networks, KNN, and Support Vector Machines, with a baseline Linear Regression for reference.

Key Insights:

- Random Forest and XGBoost delivered the most accurate and reliable predictions, outperforming simpler linear models.
- Neural Networks also performed well, capturing complex patterns.
- Accurate predictions help stakeholders price vehicles appropriately, avoid over- or under-valuing cars, and improve inventory and sales decisions.

Next Steps:

- Update the model regularly with new data.
- Explore additional features like seasonal or regional trends.
- Integrate the model into pricing and sales systems for real-time insights.

Contents

Project Summary	i
1. Introduction	1
1.1 Problem Statement	1
1.2 Stakeholder Overview	1
2. Overview of the Dataset and Exploratory Analysis	3
2.1 Dataset Characteristics	3
Type of Dataset	3
Dimensions of the Dataset	3
2.2 Variable Description and Roles	3
Variable Definitions, Types, and Roles	3
2.3 Unit of Analysis	5
Level of the Data	5
Verbal Presentation of Sample Records	5
2.4 Exploratory Data Analysis	6
2.5 Univariate Analysis	6
Univariate Analysis of the Numerical Variables	6
Univariate Analysis of the Categorical Variables	10
2.6 Bivariate Analysis (Price vs Predictors)	13
Assessment of Predictive Power	6
2.7 Data Quality Assessment	7
Outliers and Extreme Values	7
Missing Values	7
3. Predictive Modelling and Results	8
3.1 Modelling Objective and Problem Type	8
3.2 Data Partitioning Strategy	8
3.3 Performance Metrics and Model Evaluation Criteria	8
3.4 Baseline Model Development	9
3.5 Feature Engineering	9
3.6 Model Improvement and Comparative Analysis	10
Model Performance Comparison	23
Key Insights	25
3.7 Error Cost Analysis from a Stakeholder Perspective	27
4. Conclusion and Recommendations	28
	ii

4.1	Key Takeaways for Decision Makers:	28
4.2	Future Improvements and Further Research	28
References		25
Appendix		v
Codes Used:		v
	Car price histogram	v
	Car price box plot	v
	Histogram and boxplots of the numerical variables	vi
	Bar plots of the categorical variables	vii
	Correlation matrix	vii
	Scatter plot: Price vs Numerical Variables	vii
	Side-by-side box plot: Price vs Categorical Variables	viii
	ANOVA	viii
	Baseline Model – Linear Regression	ix
	Predictive Model Run	ix
	IBM SPSS Modeler	xii
	Variable Importance Rank	xii

List of Figures

Figure 2-1 A snippet into the first 5 rows and variables of the dataset	5
Figure 2-2 Car Price Distribution	7
Figure 2-3 Box-Plot of Price Variable	7
Figure 2-4 Histogram and Box-plot of Year, Door Numbers, Owners, Warranty and Credit History	8
Figure 2-5 Histogram and Box-plot of Engine Size, Car length, Car width, Monthly Mileage, Peak RPM	8
Figure 2-6 Descriptive Statistics of the Numerical Variables in the dataset	9
Figure 2-7 Frequency distribution of the categorical variables: Premium Sound, Multimedia, Bluetooth, Wheel, Sunroof, Tire Age, Numbers	10
Figure 2-8 Frequency distribution of the categorical variables: Model, Fuel Type, Transmission, Condition, Color, Cruise, Leather Seats, Heated Seats, Navigation, Insurance, Service History, Safety	11
Figure 2-9 Mode, Frequency and Variation Ratio chart of the categorical variables	12
Figure 2-10 Correlation Matrix	3
Figure 2-11 Scatter Plot: Relationship between price and Year, Number of Doors, Owners, Warranty, Credit History, Engine Size	4
Figure 2-12 Scatter Plot: Relationship between price and Car Length, Car Width, Monthly Mileage and Peak RPM	4
Figure 2-13 Side-by-side box plots for the categorical variables	5
Figure 2-14 ANOVA Table of the Categorical Variables	6
Figure 3-1 Model Comparison: RMSE with Baseline	25
Figure 3-2 Model Comparison: R square with Baseline	26

List of Tables

Table 1-1 Car Price Prediction Importance for Stakeholders	2
Table 2-1 Data Type and Data Dictionary	3
Table 3-1 Baseline Model: Linear Regression Metrics	9
Table 3-2 Model Performance Comparison	23

1. Introduction

Determining the right price for a used car is a complex task, as it hinges on a delicate balance of technical, historical, and market-driven factors. However, determining a fair and competitive price is not straightforward due to the complexity and interaction of these variables.

The objective of this project is to develop a predictive model that estimates the price of a car using a selected set of explanatory variables from the Car Price dataset. Although the original dataset contains more than twenty predictors, only ten variables are selected for modelling purposes in order to maintain practicality, interpretability, and analytical clarity. The predictors were chosen following an initial exploratory modelling process involving Machine Learning to identify the most influential variables in determining price and help with decision making without too much complexity.

1.1 Problem Statement

The central problem addressed in this report is therefore:

Can we accurately predict car prices using vehicle characteristics and historical information?

1.2 Stakeholder Overview

Several stakeholders are directly or indirectly affected by car price estimation. Especially if you look at a car dealership – the number of stakeholders involved are diverse. Everyone wants a fair pricing whether they want to sell or buy.

Given the financial and strategic implications, developing a reliable predictive model for car prices is both economically and practically significant.

Below is the list of possible stakeholders and how they will benefit from the predictive pricing.

Table 1-1 Car Price Prediction Importance for Stakeholders

Stakeholder	Importance of the Problem
Car Dealerships	Dealerships rely on precise pricing to maximize profitability while remaining competitive. Overpricing may lead to unsold inventory, while under-pricing results in reduced profit margins.
Online Marketplace	Digital platforms benefit from fair and consistent pricing models, as accurate valuations enhance customer trust and increase transaction volume.
Financial Institutions	Banks and loan providers use vehicle value estimates to determine loan amounts and assess risk exposure. Incorrect valuations may increase financial risk.
Insurance Companies	Insurance premiums and claim settlements depend on the assessed value of vehicles. Reliable price estimation ensures fair premium calculations.
Buyers and Sellers	Accurate pricing protects buyers from overpaying and helps sellers avoid undervaluing their assets.

2. Overview of the Dataset and Exploratory Analysis

2.1 Dataset Characteristics

Type of Dataset

The dataset used in this study is a structured, cross-sectional dataset consisting of used car listings. Each row represents an individual vehicle observation, and each column represents a characteristic of that vehicle.

The dataset is designed for supervised learning as we are predicting the variable Price but we can also run it for unsupervised tasks as well if we remove the price factor. The objective of this dataset is to predict the target variable (Price) using a set of explanatory variables.

Dimensions of the Dataset

The dataset contains: 2,230 observations (rows) with 32 variables (columns), including the target variable “Price”.

2.2 Variable Description and Roles

Variable Definitions, Types, and Roles

The dataset contains a mixture of numerical, categorical, ordinal, and binary variables. Price is the Target variable. The others can be grouped as follows:

Table 2-1 Data Type and Data Dictionary

Data Type	Variable	Variable Explanation
Numerical	carlength	Length of the car
	carwidth	Width of the car
	Cylinder_Numbers	Number of engine cylinders

	DoorsNum	Number of doors
	Engine_Size	Engine capacity
	Estimated_Mileage	Estimated annual mileage
	Monthly_Mileage	Average monthly mileage
	Owners	Number of previous owners
	peakrpm	Peak revolutions per minute
	TAge	Age of the tires
	Warranty	Warranty duration (months)
	Weight	Vehicle weight
	Year	Manufacturing year
Categorical	Color	Vehicle colour
	Condition	Overall condition
	Fuel_Type	Type of fuel
	Insurance	Type of insurance
	Model	Brand and model
	Service_History	Service record information
	Transmission	Manual or automatic
	Wheel	Wheel type
Binary (Yes/No)	Bluetooth	Bluetooth connectivity
	Cruise	Cruise control
	Heated_Seats	Equipped with heated seats
	Leather_Seats	Has leather seats
	Multimedia	Equipped with multimedia
	Navigation	Has navigation control

	Premium_Sound	Has premium sound system
	Sunroof	Has a sunroof

2.3 Unit of Analysis

Level of the Data

Each record in the dataset represents a single vehicle listing, including its technical specifications, physical characteristics, ownership history, optional features, and market price.

The unit of analysis is therefore an individual used car.

	Model	Year	Fuel_Type	Transmission	DoorsNum	Condition	Color	Cruise	Leather_Seats	Heated_Seats	...	Price	Cylinder_Numbers	Credit_History
0	Toyota Tundra	2017	Hybrid	Automatic	4.0	Good	Brown	Yes	No	No	...	28357.83064	five	-0.002342
1	Ford Fusion	2010	Hybrid	Automatic	2.0	Fair	Brown	No	No	No	...	20165.48038	five	-0.002341
2	Ford Escape	2005	Hybrid	Automatic	4.0	Good	Black	No	No	No	...	18352.66203	two	-0.019133
3	Nissan Altima	2008	Electric	Automatic	4.0	Fair	Brown	No	No	No	...	12523.90508	two	-0.014123
4	Nissan GT-R	2017	Petrol	Automatic	4.0	Good	Brown	No	No	No	...	12124.67706	two	-0.006006

5 rows × 32 columns

Figure 2-1 A snippet into the first 5 rows and variables of the dataset

Verbal Presentation of Sample Records

Record 1 represents a relatively modern vehicle equipped with multimedia, Bluetooth, leather seats, and navigation. It has an automatic transmission, moderate mileage, and a higher engine size, which is reflected in a comparatively higher selling price.

Record 2 represents an older vehicle with manual transmission, fewer additional features, multiple previous owners, and higher monthly mileage. These characteristics correspond to a lower market price. These examples demonstrate how combinations of structural, technical, and feature-based characteristics may influence vehicle valuation.

2.4 Exploratory Data Analysis

Exploratory Data Analysis was conducted to understand the distribution, variability, and relationships between variables prior to variable selection and modelling.

2.5 Univariate Analysis

To perform univariate analysis the dataset was divided into two parts: Numerical and Categorical including Binary (yes/no) variables.

Univariate Analysis of the Numerical Variables

Univariate analysis was performed to examine the distribution and spread of individual numerical variables. Key descriptive statistics including mean, median, standard deviation, minimum, maximum, and quartiles were calculated.

Target Variable: Price

Below is the histogram and box-plot of the target variable 'Price'. The distribution is bi-modal with a right skewness. From the histogram we see that over 30 percent of cars are priced at or below 10,000 pounds and almost 90% cars are priced at or below 30,000 pounds.

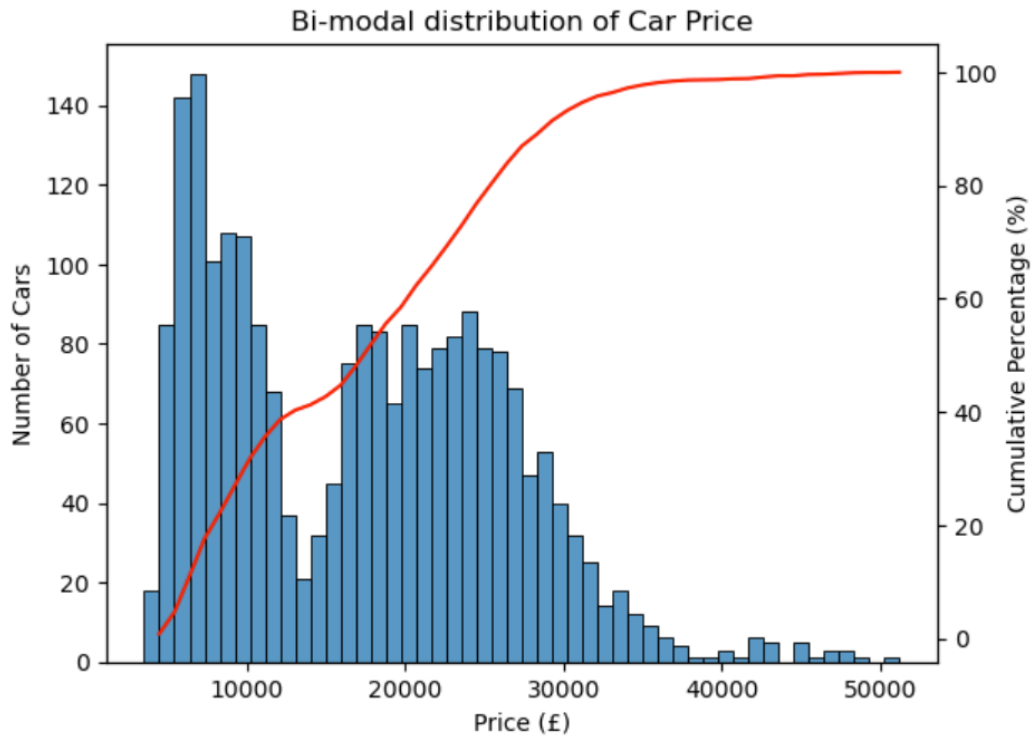


Figure 2-2 Car Price Distribution

The box-plot gives an idea about the outliers. As seen in the histogram, the box-plot shows that the dataset has a right-skewed distribution with outliers – cars priced over 45,000 pounds.

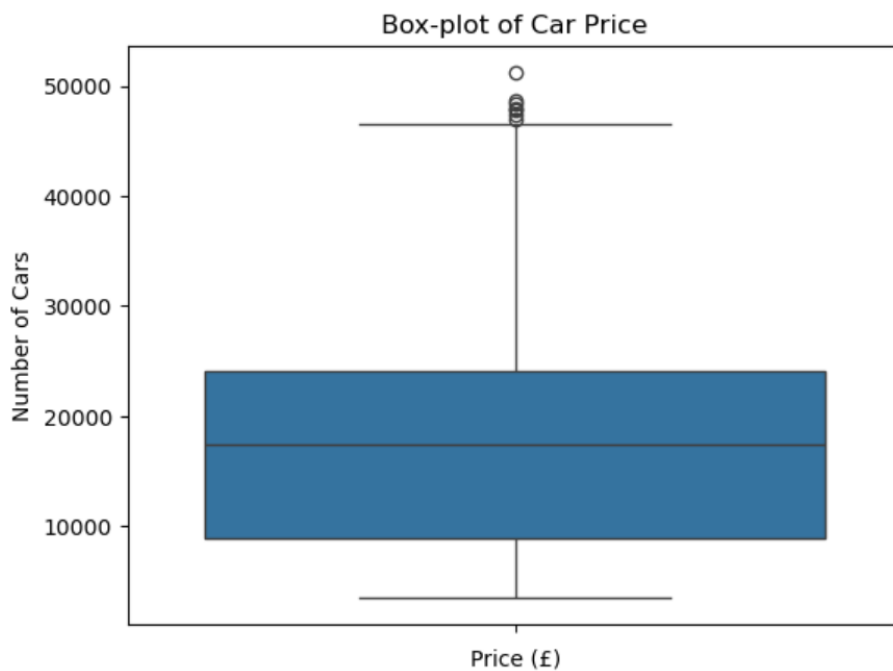


Figure 2-3 Box-Plot of Price Variable

Other Numerical Variables

Credit History is almost perfectly normally distributed but with outliers in both the sides. Year is also normally distributed with a slight right skew, meaning most of the cars listed are older. The box-plot indicates that there are no outliers – meaning the oldest car falls well within the range. Warranty has a bi-modal but almost equal distribution.

However the door numbers and owners variables interestingly show some outliers with a very large value – suggesting incorrect record.

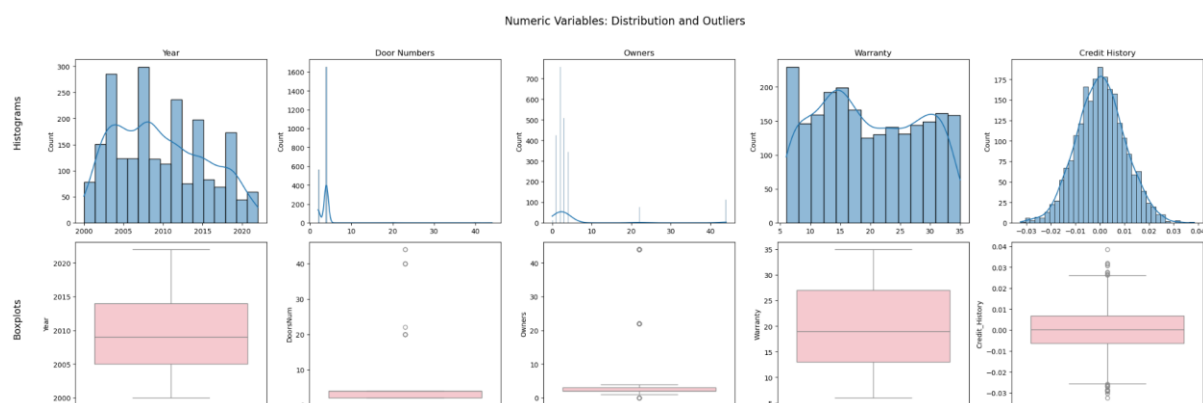


Figure 2-4 Histogram and Box-plot of Year, Door Numbers, Owners, Warranty and Credit History

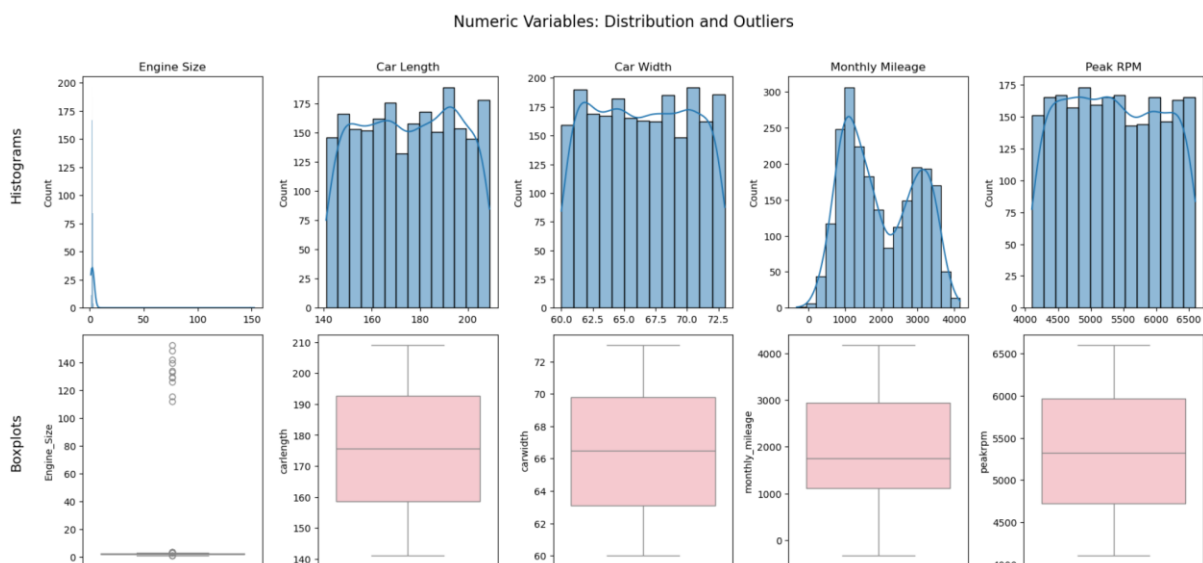


Figure 2-5 Histogram and Box-plot of Engine Size, Car length, Car width, Monthly Mileage, Peak RPM

Car length, Car Width and Peak RPM variables have a flat distribution with no outliers. Monthly Mileage value distribution also falls within the inter quartile range. However, Engine Size clearly has some unrealistic values – suggesting incorrect records.

Measures of Central Tendency and Dispersion

The following is a descriptive statistics of the numerical variables. They are performed to examine the distribution and spread of individual numerical variables. Key descriptive statistics including mean, median, standard deviation, minimum, maximum, and quartiles were calculated.

	Year	DoorsNum	Owners	Warranty	Price	Credit_History	Engine_Size	carlength	carwidth	monthly_mileage	peakrpm
count	2230.000000	2230.000000	2230.000000	2230.000000	2230.000000	2230.000000	2230.000000	2230.000000	2230.000000	2230.000000	2230.000000
mean	2009.702242	3.641256	5.117489	19.735874	17241.511048	0.000360	2.931071	175.439552	66.477309	1986.593741	5343.346637
std	5.676737	2.314931	9.643330	8.419290	8961.471367	0.009869	9.195792	19.677468	3.770973	994.408364	722.580399
min	2000.000000	2.000000	0.000000	6.000000	3527.516896	-0.032413	1.000000	141.000000	60.000000	-326.426931	4100.000000
25%	2005.000000	2.000000	2.000000	13.000000	8897.953205	-0.006230	2.020221	158.500000	63.100000	1116.573719	4721.250000
50%	2009.000000	4.000000	2.000000	19.000000	17393.385485	0.000253	2.284103	175.600000	66.500000	1758.514301	5324.500000
75%	2014.000000	4.000000	3.000000	27.000000	24067.296380	0.006728	2.553892	192.575000	69.800000	2940.711792	5969.000000
max	2022.000000	44.000000	44.000000	35.000000	51219.220660	0.038527	152.348978	209.000000	73.000000	4167.599834	6600.000000

Figure 2-6 Descriptive Statistics of the Numerical Variables in the dataset

Years: The majority of vehicles fall within the used car segment, with an average age of approximately 13 years. This confirms that the dataset primarily represents the secondary vehicle market.

Owners: While the average number of doors is reasonable, the maximum value of 44 is unrealistic for passenger vehicles. This suggests the presence of extreme values or data entry errors.

Warranty: Warranty duration appears reasonable and consistent with typical used car warranty offerings.

Credit History: The variable is centered around zero with limited dispersion. This suggests it may be standardized or scaled and may contribute limited variation to the model.

Engine Size: The maximum value is clearly unrealistic for engine capacity, indicating significant outliers or possible scaling inconsistencies. This variable requires careful cleaning before modelling.

Monthly Mileage: Negative mileage values are not logically possible, indicating data quality issues. These values must be corrected or removed before model development.

Peak RPM: These values appear consistent with typical vehicle specifications and do not immediately indicate anomalies.

Several variables demonstrated substantial dispersion and potential extreme values, particularly DoorsNum, Owners, Engine_Size, and Monthly_Mileage.

Univariate Analysis of the Categorical Variables

This section presents the measures of central tendency and dispersion for the categorical variables included in the analysis. Since categorical variables do not possess numerical magnitude, the mean and standard deviation are not appropriate measures. Instead, the mode is used as the measure of central tendency, while frequency distributions and proportional shares are used to assess dispersion.

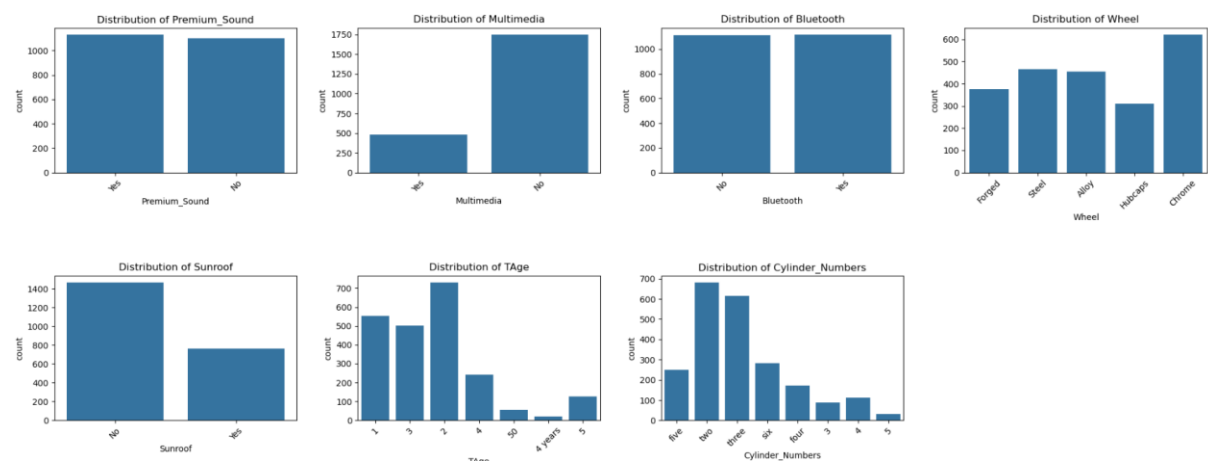


Figure 2-7 Frequency distribution of the categorical variables: Premium Sound, Multimedia, Bluetooth, Wheel, Sunroof, Tire Age, Numbers

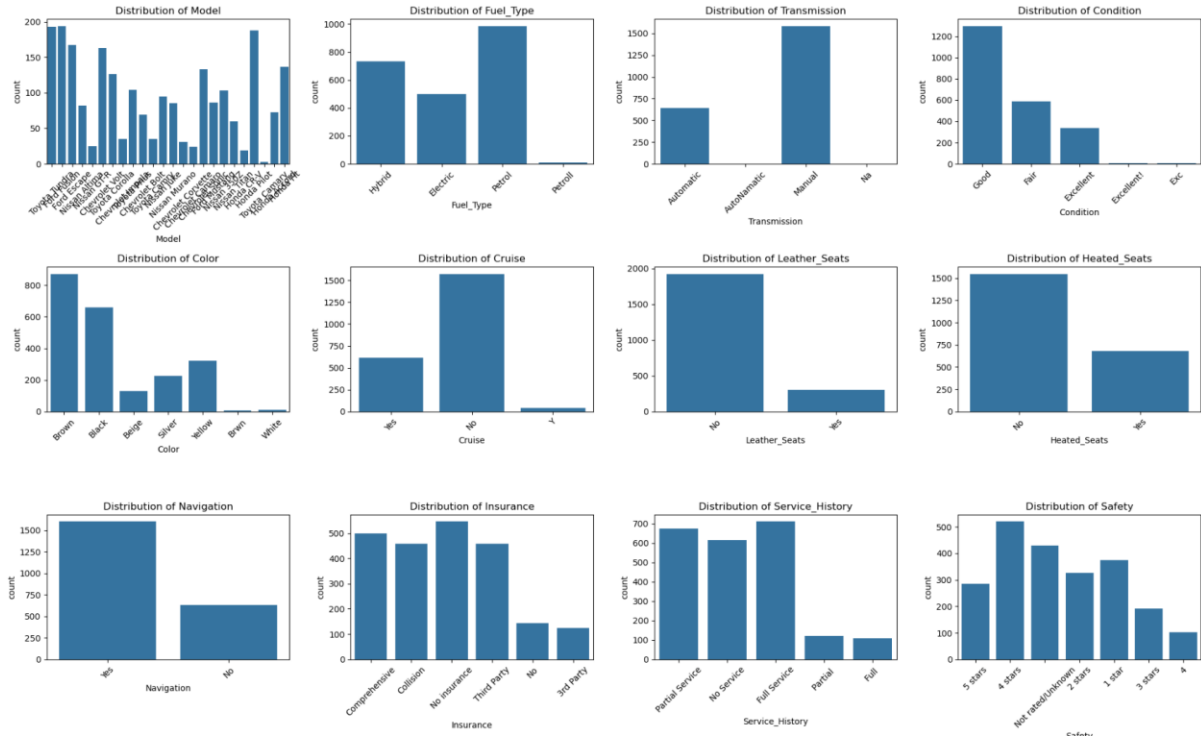


Figure 2-8 Frequency distribution of the categorical variables: Model, Fuel Type, Transmission, Condition, Color, Cruise, Leather Seats, Heated Seats, Navigation, Insurance, Service History, Safety

Measures of Central Tendency and Dispersion

The central tendency of categorical variables was assessed using the **mode**, while dispersion was evaluated using the **variation ratio**, defined as:

$$VR = 1 - \frac{f_{mode}}{n}$$

where f_{mode} is the frequency of the most common category and n is the total number of observations. A lower variation ratio indicates higher concentration around a single category, whereas higher values indicate greater diversity.

Below is the breakdown of each categorical variable's mode, frequency and variation ratio.

	Variable	Mode	Mode_Frequency	Variation_Ratio
0	Model	Ford Fusion	194	0.913
1	Fuel_Type	Petrol	986	0.558
2	Transmission	Manual	1584	0.290
3	Condition	Good	1296	0.419
4	Color	Brown	872	0.609
5	Cruise	No	1573	0.295
6	Leather_Seats	No	1925	0.137
7	Heated_Seats	No	1548	0.306
8	Navigation	Yes	1600	0.283
9	Insurance	No insurance	546	0.755
10	Service_History	Full Service	710	0.682
11	Safety	4 stars	520	0.767
12	Premium_Sound	Yes	1130	0.493
13	Multimedia	No	1748	0.216
14	Bluetooth	Yes	1116	0.500
15	Wheel	Chrome	621	0.722
16	Sunroof	No	1467	0.342
17	TAge	2	730	0.673
18	Cylinder_Numbers	two	681	0.695

Figure 2-9 Mode, Frequency and Variation Ratio chart of the categorical variables

The dataset shows substantial diversity in Model (VR = 0.913), Safety (VR = 0.767), Insurance (VR = 0.755), and Wheel Type (VR = 0.722), suggesting considerable heterogeneity across vehicles. Such variability may enhance their potential predictive contribution.

In contrast, features such as Leather Seats (VR = 0.137) and Multimedia (VR = 0.216) exhibit strong concentration around a single category, indicating limited dispersion. These variables may have weaker discriminatory power in predictive modelling due to their lack of variability. Moderate dispersion is observed in variables such as Fuel Type (VR = 0.558), Condition (VR = 0.419), and Transmission (VR = 0.290), suggesting balanced category representation.

Overall, the dataset contains both highly diverse and highly concentrated categorical features, which may influence their relative importance in price prediction.

2.6 Bivariate Analysis (Price vs Predictors)

While univariate analysis provides insight into the distribution and variability of each variable independently, it does not explain their relationship with the target variable, Price. Therefore, the next section examines the association between each predictor and price through bivariate analysis to assess their predictive strength and practical relevance.

Price and the Numerical Variables:

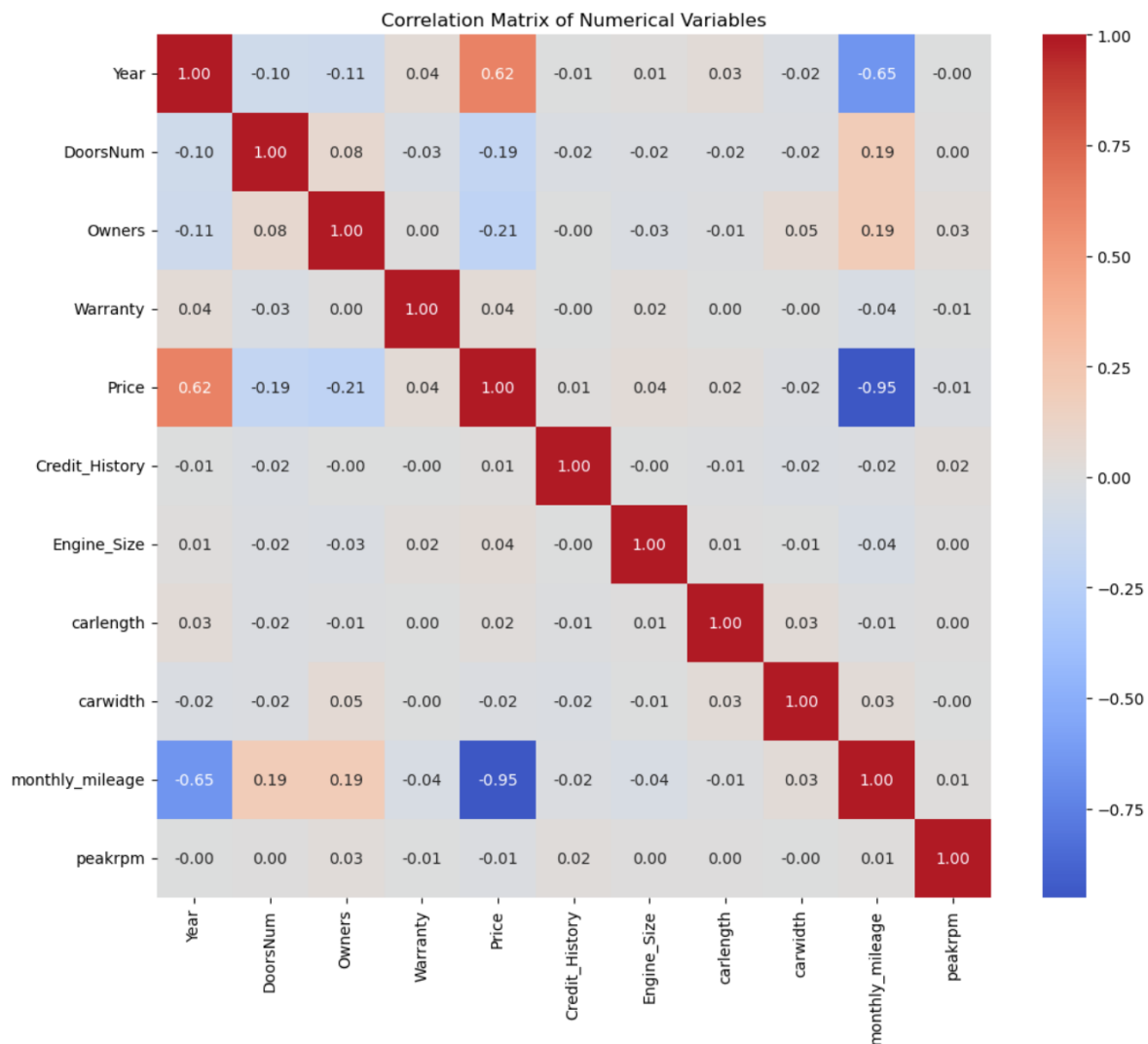


Figure 2-10 Correlation Matrix

The matrix shows that, according to the Pearson correlation, Year, Monthly Mileage and Owners have a correlation to Price (above 20%).

The scatter plots in the next page reinforce the findings.

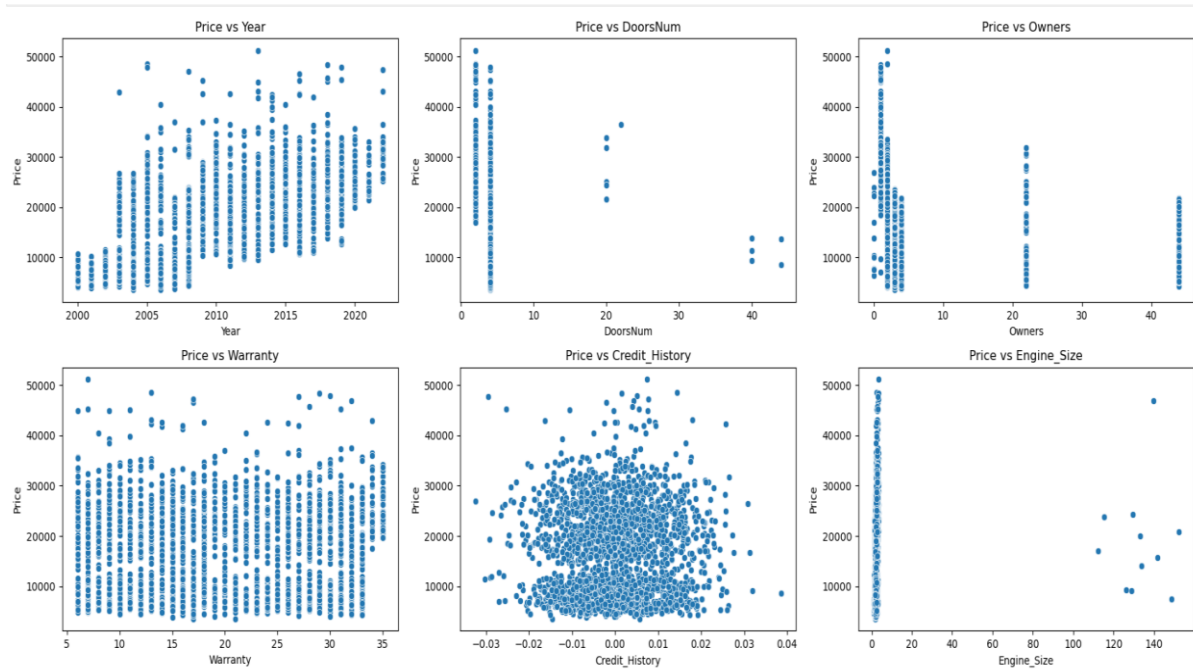


Figure 2-11 Scatter Plot: Relationship between price and Year, Number of Doors, Owners, Warranty, Credit History, Engine Size

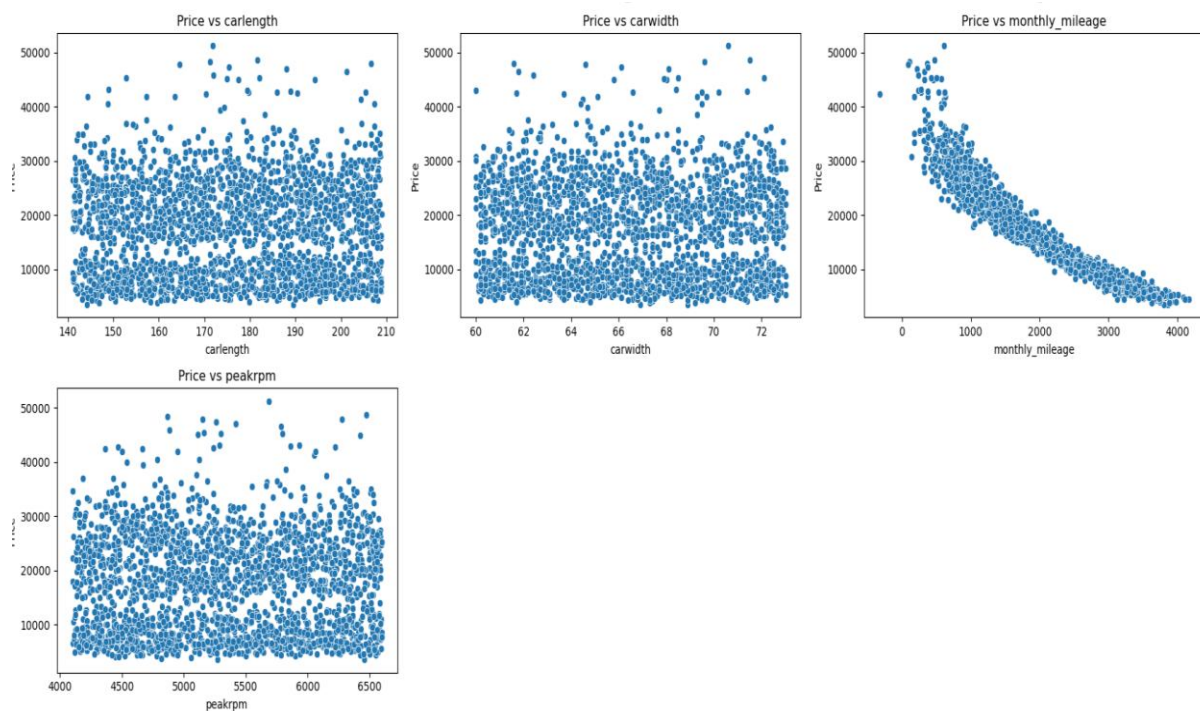
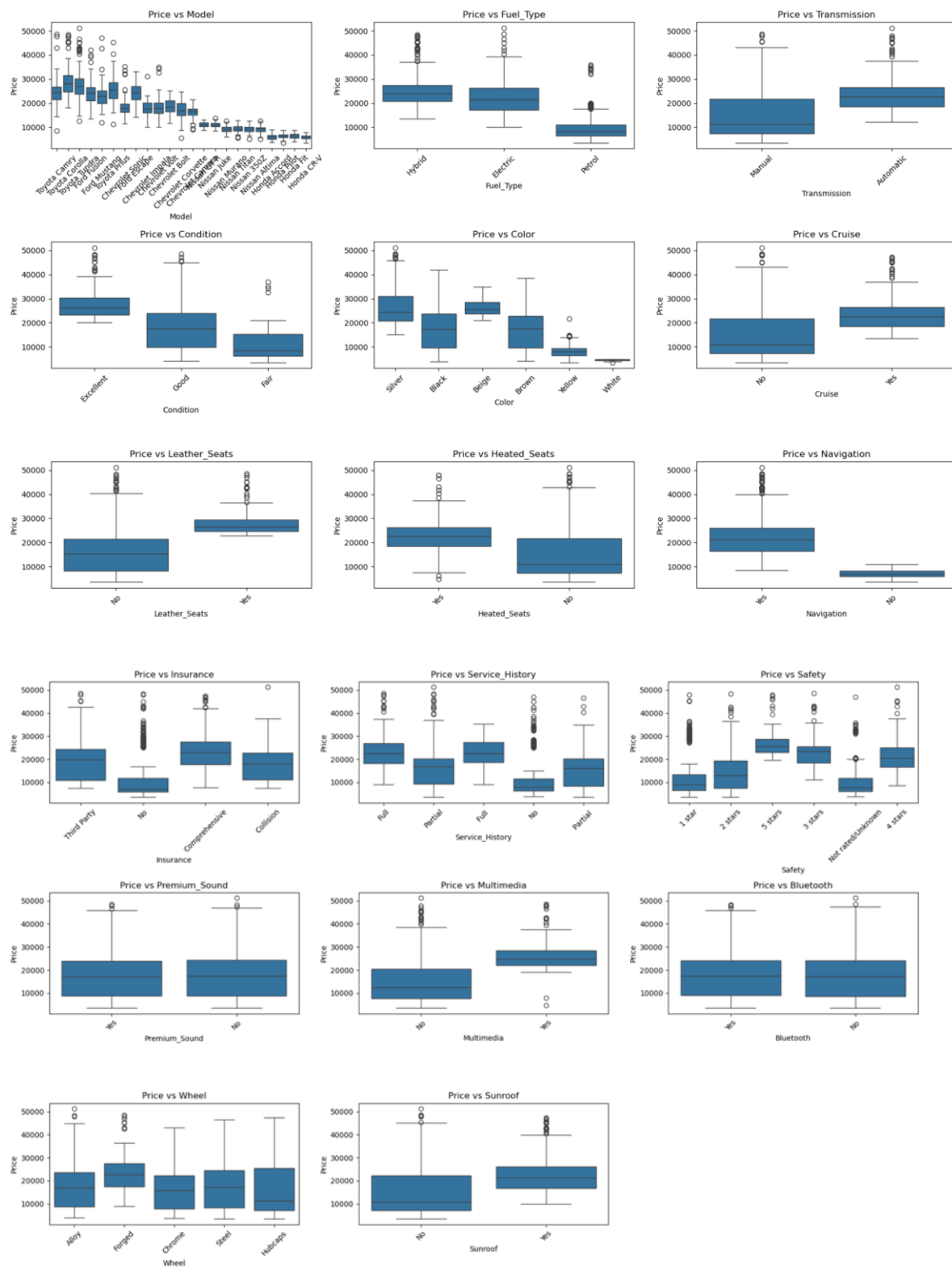


Figure 2-12 Scatter Plot: Relationship between price and Car Length, Car Width, Monthly Mileage and Peak RPM

Only Year, Monthly Mileage and Owners have a visible correlation to Price.



While the graphs reveal how certain features of a car prone to have higher price e.g. leather seats, fuel types, in order to get a clearer picture and assess the predictive power of the variables we performed ANOVA.

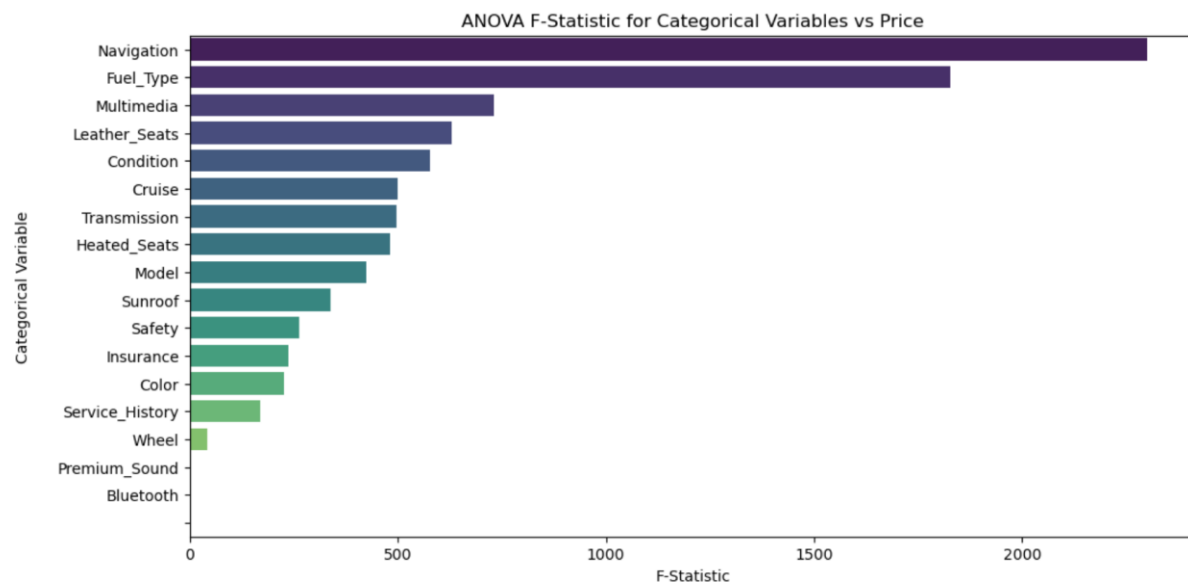


Figure 2-14 ANOVA Table of the Categorical Variables

ANOVA was used to assess whether mean Price differs across categories. Variables with high F-statistics and $p < 0.05$ indicate strong predictive potential. According to the analysis strong predictors are Navigation (F = 2301), Fuel_Type (F = 1827), Multimedia (F = 730), Leather_Seats (F = 630), Condition, Cruise, Transmission, Heated_Seats, Model, Sunroof, Safety, Insurance, Color, Service_History.

Assessment of Predictive Power

The Pearson Correlation Matrix helped prioritize the candidate features from the numerical variables that affect the price more. Whereas, these ANOVA results helped prioritize candidate features for predictive modeling. Variables such as Navigation, Fuel_Type, Multimedia, and

Leather_Seats are likely to be the most influential, while Premium_Sound and Bluetooth may be excluded. The third section of this report implements regression and machine learning models to formally assess predictive power, select the top predictors, and evaluate model performance.

2.7 Data Quality Assessment

Outliers and Extreme Values

The exploratory analysis revealed several anomalies:

- Unrealistic maximum values (e.g., 44 doors, 44 owners).
- Implausible engine size value (152).
- Negative mileage observations.
- Weight, Credit History, Tire Age seeing high number of outliers – values going past the inter quartile range (IQR).

Aside from the unrealistic values we are keeping them as it is; as outliers are not errors, but represent rare but possible car types (e.g., very expensive or very large cars).

Missing Values

The dataset contained a small number of missing values:

Fuel_Type: 3, Transmission: 7, DoorsNum: 7, Color: 2, Weight: 1

These missing values represent less than 1% of the total dataset (2,230 observations). Given their minimal proportion, and after variable selection (where most affected variables were not retained except Transmission), the corresponding records were removed to maintain dataset consistency without introducing imputation bias.

After removal of missing records, the dataset remained sufficiently large for robust modelling.

3. Predictive Modelling and Results

3.1 Modelling Objective and Problem Type

The objective of this analysis is to predict the market Price (£) of cars based on selected vehicle characteristics. The problem is therefore a **supervised regression** task, as the target variable, Price, is continuous. Predicting Price allows stakeholders to make informed pricing, sales, and inventory decisions. However, we have also applied un-supervised models like the neural network on the dataset to see how the model performs and if they are better predictor of price. The selected predictors were identified using a combination of **IBM SPSS Modeler's feature selection node, ANOVA, and Random Forest importance scores**. Thirteen variables appeared consistently across methods, from which ten were chosen based on domain relevance and predictive power:

Monthly_Mileage, Fuel_Type, Service_History, Owners, Safety, Wheel, Insurance, Condition, Leather_Seats, Transmission

3.2 Data Partitioning Strategy

The dataset was split into training (80%) and testing (20%) sets. This ensures that:

- The model learns patterns from the training set.
- The testing set evaluates generalisation performance on unseen data, reducing the risk of overfitting.
- Random splitting maintains representative distributions of Price and categorical variables across both sets.

3.3 Performance Metrics and Model Evaluation Criteria

Since Price is continuous, the following metrics were used:

- **Mean Squared Error (MSE):** average squared difference between predicted and actual Price. Penalizes large errors.
- **Root Mean Squared Error (RMSE):** square root of MSE; same units as Price (£).
- **Mean Absolute Error (MAE):** average absolute difference between predicted and actual Price; robust to outliers.
- **R-squared (R^2):** proportion of variance explained by the model; higher values indicate better fit.

Rationale for choice: RMSE is preferred as it is interpretable in Price units and sensitive to large deviations, which is important for pricing accuracy. (Hyndman, 2018)

3.4 Baseline Model Development

A baseline linear regression model was developed using the ten selected predictors. This provides a reference point to evaluate improvements achieved by more advanced models.

Table 3-1 Baseline Model: Linear Regression Metrics

Model	Linear Regression (Baseline)
Hyperparameters	None
Training RMSE (£)	2534.47
Testing RMSE (£)	2156.21
Training R^2	0.92
Testing R^2	0.94
Notes	Simple linear model; good general fit but cannot capture non-linearities

3.5 Feature Engineering

Key feature engineering steps included:

1. **Handling NAN values:**

- Numerical variables → median imputation
- Categorical variables → mode imputation

2. Removing the Outliers

- The extreme/implausible values as discovered in the Outliers section

3. Encoding categorical variables: One-hot encoding for Fuel_Type, Transmission, Service_History, etc.

4. Scaling numerical features: Required for LSVM, KNN, and Neural Network models.

These steps improved **model robustness** and **predictive performance**, especially for algorithms sensitive to scale or sparsity.

Please note that, initially, as the records with missing values were less than 1% of the dataset – they were deleted. However, the NaN values were handled with the steps mentioned above.

3.6 Model Improvement and Comparative Analysis

The following models were applied to the training data:

- **Random Forest Regressor (RF)** – ensemble method, robust to non-linearities (Breiman, 2001)
- **Linear Support Vector Machine (LSVM)** – regression using margin maximization
- **XGBoost Regressor** – gradient boosting, handles missing values, strong for tabular data (Anon., 2024)
- **K-Nearest Neighbors (KNN)** – distance-based prediction (Hastie, 2009)
- **Neural Network (NN)** – multi-layer perceptron for non-linear patterns (Goodfellow, 2016)
- **Logistic Regression** – included for completeness but not suitable for continuous Price; omitted in final regression comparison

Performance of the Random Forest and XGBoost along with Neural Network were the best as we will see in the Model Performance Comparison.

Random Forest (RF)

- RF is an **ensemble method** that builds multiple decision trees on random subsets of data and features.
- Each tree votes for a prediction, and the final output is the average of all trees (for regression).
- **Why it performed well here:**
 - Handles non-linear relationships and interactions between variables.
 - Robust to outliers and missing values (after imputation).
 - Reduces overfitting via averaging multiple trees. (Breiman, 2001)
- In our dataset, RF captured patterns across features like Monthly_Mileage, Condition, Fuel_Type, and Owners, which interact in non-linear ways.

XGBoost (Extreme Gradient Boosting)

- XGBoost is a **gradient boosting ensemble method** that builds trees sequentially, where each tree tries to correct the errors of the previous one.
- Optimized for speed and accuracy, includes **regularization** to prevent overfitting.
- **Why it performed well:**
 - Excellent for tabular datasets with a mix of numerical and categorical variables.
 - Can handle complex feature interactions.
 - Uses shrinkage (learning rate) and tree depth control to balance bias and variance. (Anon., 2024)
- In our dataset, XGBoost learned subtle patterns that RF might average out, which explains why it also gave high predictive accuracy.

Neural Network (Multi-Layer Perceptron)

- A neural network consists of **layers of interconnected nodes (“neurons”)** that transform input features into outputs.
- **Why it performed well here:**
 - Can model complex non-linear relationships between features and target (Price).
 - Good at capturing interactions between multiple variables simultaneously.
 - Flexible: we can adjust the number of hidden layers and neurons to balance underfitting vs overfitting.
- In the dataset, the NN learned patterns across variables like Monthly_Mileage, Condition, Fuel_Type, and Safety, producing high **testing R^2 (~0.95)**.
- **Caution:** Neural Networks can **overfit small datasets** if too many neurons/layers are used. That’s why scaling features and using a reasonable architecture (e.g., 2 hidden layers of 50 neurons) is important. (Goodfellow, 2016)

Key takeaway: Both RF and XGBoost are ensemble tree methods. RF averages trees independently, whereas XGBoost builds trees sequentially to minimize errors. This makes them strong performers for predicting car prices with mixed-type features and non-linear effects.

Neural Network has strong accuracy, captures non-linear interactions, but less interpretable.

Model Performance Comparison

The predictive performance of the baseline model and all regression models with multiple hyperparameter configurations is summarised below:

Table 3-2 Model Performance Comparison

Model	Hyperparameters	Training RMSE (£)	Testing RMSE (£)	Training R ²	Testing R ²	Notes
Linear Regression (Baseline)	None	2534.47	2156.21	0.92	0.94	Simple linear model; good general fit but cannot capture non-linearities
Random Forest	n_estimators=150, max_depth=15	820.82	1685.08	0.99	0.96	Best balance of training fit and generalisation
Random Forest	n_estimators=100, max_depth=10	1109.74	1706.97	0.98	0.96	Good overall performance
Random Forest	n_estimators=50, max_depth=5	1919.03	1784.24	0.95	0.96	Underfit model, simpler trees
XGBoost	n_estimators=50, max_depth=3	1906.75	1797.66	0.96	0.96	Small trees, underfitting
XGBoost	n_estimators=100, max_depth=6	896.41	1872.05	0.99	0.96	Strong performance, balanced

XGBoost	n_estimators=150, max_depth=9	475.65	1887.94	1.00	0.95	Slight overfitting, high training fit
Neural Network	(50,50)	2276.47	1995.03	0.94	0.95	Good fit, slight overfitting
Neural Network	(25,25)	6472.64	6524.03	0.48	0.45	Too small network, underfitting
Neural Network	(100,50)	19254.53	19052.70	-3.59	-3.65	Oversized network, diverging
KNN	n_neighbors=5, weights='distance'	0.00	3353.09	1.00	0.86	Training overfit, sensitive to scaling
KNN	n_neighbors=7, weights='uniform'	3422.96	3447.47	0.86	0.85	Moderate performance
KNN	n_neighbors=3, weights='uniform'	2904.56	3541.66	0.90	0.84	Slightly overfit
LSVM	C=2.0, epsilon=0.05	8819.65	8660.81	0.04	0.04	Poor fit, can't capture non-linearity
LSVM	C=1.0, epsilon=0.1	8905.13	8749.46	0.02	0.02	Similar poor performance
LSVM	C=0.5, epsilon=0.2	8948.08	8793.93	0.01	0.01	Slightly underfit

Key Insights

□ Baseline Performance:

- The linear regression baseline provides reasonable predictions (Testing RMSE $\approx 2,156$ £, $R^2=0.94$) and serves as a benchmark to evaluate more complex models.

Below are the graphs that shows the best performing predictive models against the Baseline threshold.

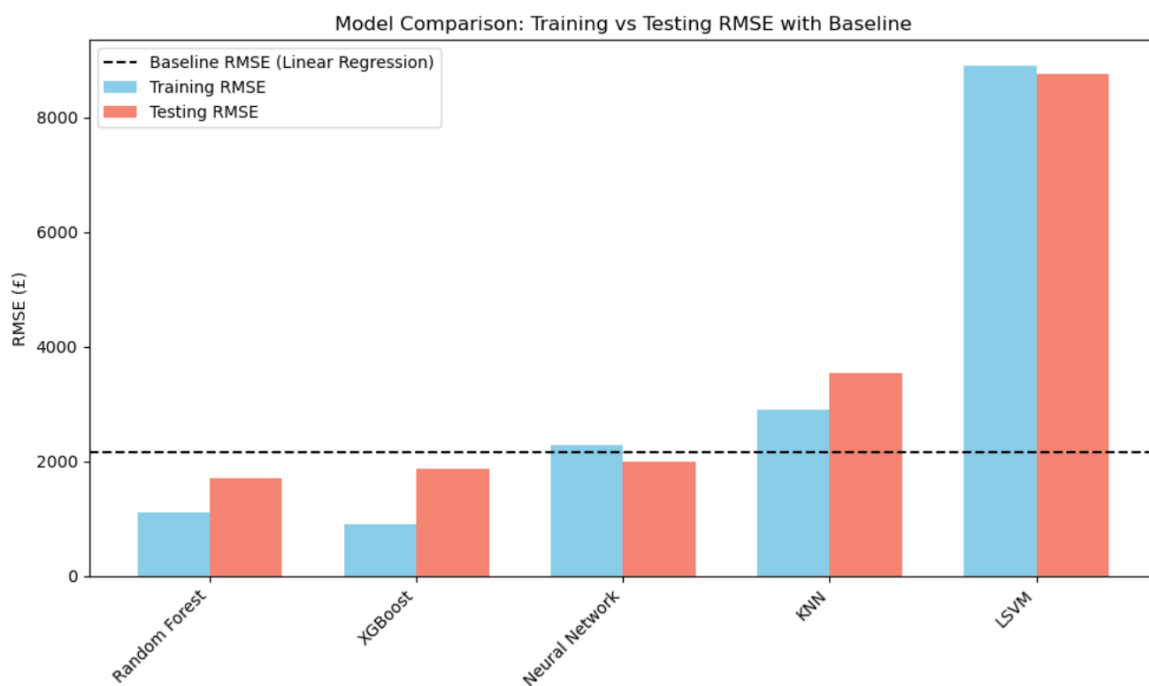


Figure 3-1 Model Comparison: RMSE with Baseline

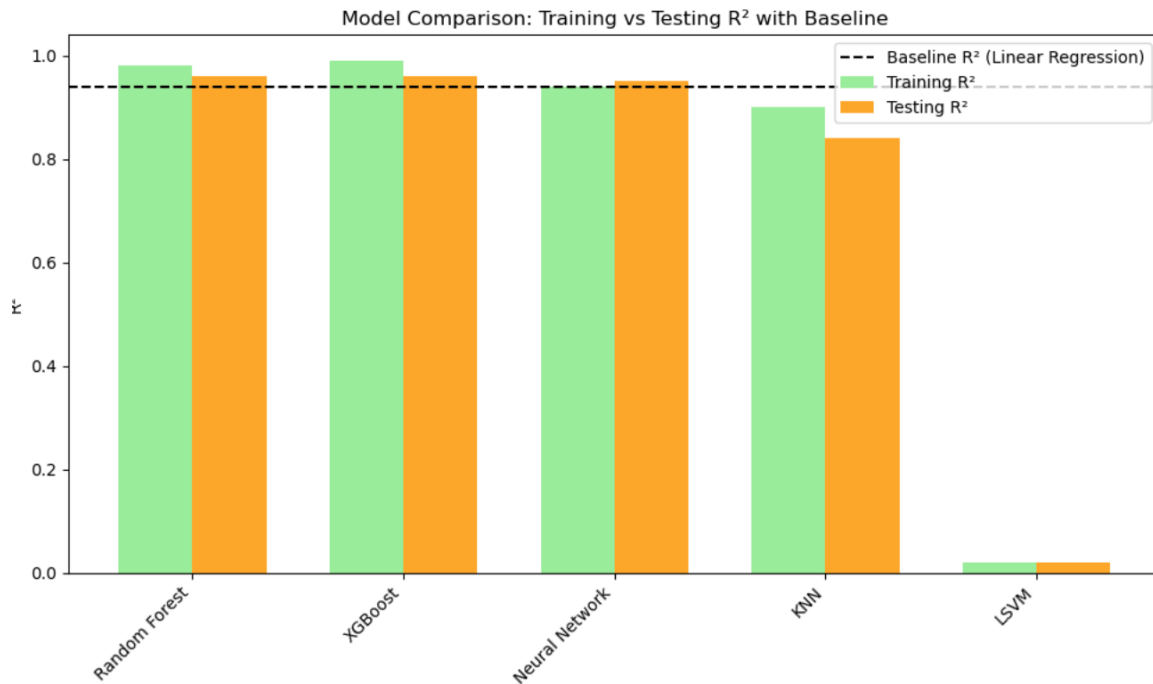


Figure 3-2 Model Comparison: R square with Baseline

□ Best Performing Models:

- **Random Forest** ($n_{\text{estimators}}=150$, $\text{max_depth}=15$) has the lowest testing RMSE (1,685 £) and high R^2 (0.96).
- **XGBoost and Neural Network (50,50)** also perform well ($R^2 \geq 0.95$), capturing non-linear relationships effectively.

□ Moderate Models:

- **KNN** shows moderate performance ($R^2 \approx 0.85$) and is sensitive to outliers and scaling.

□ Poor Models:

- **LSVM** performs poorly ($R^2 \approx 0.02$) and fails to capture the complex non-linear patterns in the dataset.

□ Training vs Testing Fit:

- Ensemble methods balance training fit and generalisation.
- Neural Network slightly overfits but still provides high accuracy.

3.7 Error Cost Analysis from a Stakeholder Perspective

Following is the error cost analysis from a stakeholders perspective.

- **Over-prediction of Price:** May lead to unsold inventory or reduced sales velocity, directly affecting revenue.
- **Under-prediction of Price:** Can undervalue cars, resulting in potential profit loss.
- **Worst-case error:** Over-predicting high-value cars is most costly, as it can create unrealistic pricing expectations.

Selecting models like **Random Forest or XGBoost** minimises these errors and balances both accuracy and practical utility for stakeholders. Ensemble methods and moderate Neural Networks are recommended for deployment due to high accuracy, low risk of extreme mispredictions, and interpretability.

4. Conclusion and Recommendations

This project aimed to predict car prices using a combination of vehicle features such as mileage, fuel type, condition, and safety ratings. By analysing past sales data, we were able to identify which features most strongly influence a car's price and use this information to make accurate predictions.

4.1 Key Takeaways for Decision Makers:

- Cars with lower mileage, higher safety ratings, and full service histories tend to have higher market value.
- Features like leather seats, fuel type, and condition also affect price, but to a lesser degree.
- Predictive models allow pricing to be set more accurately, helping to avoid undervaluing or overpricing vehicles.

4.2 Future Improvements and Further Research

- Explore additional predictive methods to capture subtle patterns in price changes.
- Regularly update the model as new sales data becomes available to maintain accuracy.
- Develop a dashboard for real-time price predictions to support sales and inventory decisions.
- Use similar methods to forecast demand, identify high-value vehicles, or optimise inventory purchases.

References

Anon., 2024. Implementation and Performance Comparison of Gradient Boosting Algorithms for Tabular Data Classification. *Deep Learning and Visual Artificial Intelligence*, pp. 461-479.

Breiman, L., 2001. Random Forests. *Machine Learning*, 45(1), p. 5–32.

Goodfellow, I. B. Y. & C. A., 2016. *Neural Networks / Deep Learning*. s.l.:MIT Press.

Hastie, T. T. R. & F. J., 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2 ed. s.l.:s.n.

Hyndman, R. J. & A. G., 2018. *Principles and Practice*. s.l.:OTexts.

Appendix

Codes Used:

Car price histogram

```
# Importing the relevant packages for data visualisation
import matplotlib as plt
import matplotlib.pyplot as plt
import seaborn as sns

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create figure
fig, ax1 = plt.subplots()

# Histogram (frequency)
sns.histplot(cleaned_data["Price"],
             bins=50,
             kde=False,
             ax=ax1)

ax1.set_xlabel("Price (£)")
ax1.set_ylabel("Number of Cars")
ax1.set_title("Bi-modal distribution of Car Price")

# Create cumulative percentage
counts, bin_edges = np.histogram(cleaned_data["Price"], bins=50)
cumulative = np.cumsum(counts)
cumulative_percentage = cumulative / cumulative[-1] * 100

# Secondary axis
ax2 = ax1.twinx()
ax2.plot(bin_edges[1:], cumulative_percentage, color='red')
ax2.set_ylabel("Cumulative Percentage (%)")

plt.show()
```

Car price box plot

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create figure
fig, ax1 = plt.subplots()

# Histogram (frequency)
sns.boxplot(cleaned_data["Price"])

ax1.set_xlabel("Price (£)")
ax1.set_ylabel("Number of Cars")
ax1.set_title("Box-plot of Car Price")

plt.show()
```

Histogram and boxplots of the numerical variables

```
#List of numeric columns
num_cols = ['Engine_Size', 'carlength', 'carwidth', 'monthly_mileage', 'peakrpm' ]

#Column Labels
ncol_names = ['Engine Size', 'Car Length', 'Car Width', 'Monthly Mileage', 'Peak RPM']

# 2 rows - for distribution and histogram, 8 columns for 8 variables
fig, axes = plt.subplots(2, 5, figsize=(17, 8))

for i, (col, name) in enumerate(zip(num_cols, ncol_names)):
    # Histogram in first row
    sns.histplot(df_n[col], kde=True, ax=axes[0, i])
    axes[0, i].set_title(f'{name}')
    axes[0, i].set_xlabel('') # Removing x-axis titles to declutter the charts

    # Boxplot in second row
    sns.boxplot(y=df_n[col], ax=axes[1, i], color='pink')

# Adding a single title for the whole grid
fig.suptitle('Numeric Variables: Distribution and Outliers', fontsize=16, y=1.02)

# Adding row Labels (y-position adjusted)
fig.text(-0.02, 0.75, 'Histograms', fontsize=14, va='center', rotation = 'vertical')
fig.text(-0.02, 0.25, 'Boxplots', fontsize=14, va='center', rotation='vertical')

plt.tight_layout()
plt.show()
```

```
#List of numeric columns
num_cols = ['Year', 'DoorsNum', 'Owners', 'Warranty', 'Credit_History']

#Column Labels
ncol_names = ['Year', 'Door Numbers', 'Owners', 'Warranty', 'Credit History']

# 2 rows - for distribution and histogram, 8 columns for 8 variables
fig, axes = plt.subplots(2, 5, figsize=(24, 8))

for i, (col, name) in enumerate(zip(num_cols, ncol_names)):
    # Histogram in first row
    sns.histplot(df_n[col], kde=True, ax=axes[0, i])
    axes[0, i].set_title(f'{name}')
    axes[0, i].set_xlabel('') # Removing x-axis titles to declutter the charts

    # Boxplot in second row
    sns.boxplot(y=df_n[col], ax=axes[1, i], color='pink')

# Adding a single title for the whole grid
fig.suptitle('Numeric Variables: Distribution and Outliers', fontsize=16, y=1.02)

# Adding row Labels (y-position adjusted)
fig.text(-0.02, 0.75, 'Histograms', fontsize=14, va='center', rotation = 'vertical')
fig.text(-0.02, 0.25, 'Boxplots', fontsize=14, va='center', rotation='vertical')

plt.tight_layout()
plt.show()
```

Bar plots of the categorical variables

```
import matplotlib.pyplot as plt
import seaborn as sns

n_cols = 4
n_rows = (len(categorical_vars) + n_cols - 1) // n_cols

fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, n_rows*4))
axes = axes.flatten()

for i, col in enumerate(categorical_vars):
    sns.countplot(data=cleaned_data, x=col, ax=axes[i])
    axes[i].set_title(f'Distribution of {col}')
    axes[i].tick_params(axis='x', rotation=45)

# Remove empty subplots if any
for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```

Correlation matrix

```
import seaborn as sns
import matplotlib.pyplot as plt

# Select numerical variables
numerical_vars = cleaned_data.select_dtypes(include=['int64', 'float64'])

# Correlation matrix
corr_matrix = numerical_vars.corr()

# Plot heatmap
plt.figure(figsize=(12,10))
sns.heatmap(corr_matrix,
            annot=True,
            fmt=".2f",
            cmap="coolwarm",
            center=0)

plt.title("Correlation Matrix of Numerical Variables")
plt.show()
```

Scatter plot: Price vs Numerical Variables

```
numerical_predictors = numerical_vars.columns.drop("Price")

n_cols = 3
n_rows = (len(numerical_predictors) + n_cols - 1) // n_cols

fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, n_rows*4))
axes = axes.flatten()

for i, col in enumerate(numerical_predictors):
    sns.scatterplot(data=cleaned_data, x=col, y="Price", ax=axes[i])
    axes[i].set_title(f"Price vs {col}")

# Remove empty plots
for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```

Side-by-side box plot: Price vs Categorical Variables

```
categorical_vars = cleaned_data.select_dtypes(include=['object']).columns

n_cols = 3
n_rows = (len(categorical_vars) + n_cols - 1) // n_cols

fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, n_rows*4))
axes = axes.flatten()

for i, col in enumerate(categorical_vars):
    sns.boxplot(data=cleaned_data, x=col, y="Price", ax=axes[i])
    axes[i].set_title(f"Price vs {col}")
    axes[i].tick_params(axis='x', rotation=45)

for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```

ANOVA

Anova table

```
from scipy.stats import f_oneway
import pandas as pd

anova_results = []

for col in categorical_vars:
    groups = [group["Price"].values for name, group in cleaned_data.groupby(col)]
    if len(groups) > 1: # At Least two groups needed
        f_stat, p_val = f_oneway(*groups)
        anova_results.append((col, f_stat, p_val))

anova_df = pd.DataFrame(anova_results, columns=["Variable", "F_statistic", "p_value"])
anova_df = anova_df.sort_values(by="F_statistic", ascending=False)
anova_df
```

Anova Chart

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,6))
sns.barplot(data=anova_df, x="F_statistic", y="Variable", palette="viridis")
plt.title("ANOVA F-Statistic for Categorical Variables vs Price")
plt.xlabel("F-Statistic")
plt.ylabel("Categorical Variable")
plt.show()
```


Baseline Model – Linear Regression

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

baseline_model = LinearRegression()
baseline_model.fit(X_train, y_train)

y_train_pred = baseline_model.predict(X_train)
y_test_pred = baseline_model.predict(X_test)

baseline_results = {
    "Model": "Linear Regression (Baseline)",
    "Hyperparameters": "None",
    "Training MSE": round(mean_squared_error(y_train, y_train_pred),2),
    "Testing MSE": round(mean_squared_error(y_test, y_test_pred),2),
    "Training RMSE": round(np.sqrt(mean_squared_error(y_train, y_train_pred)),2),
    "Testing RMSE": round(np.sqrt(mean_squared_error(y_test, y_test_pred)),2),
    "Training MAE": round(mean_absolute_error(y_train, y_train_pred),2),
    "Testing MAE": round(mean_absolute_error(y_test, y_test_pred),2),
    "Training R²": round(r2_score(y_train, y_train_pred),2),
    "Testing R²": round(r2_score(y_test, y_test_pred),2)
}

baseline_results
```

Predictive Model Run

Performance Values

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
from sklearn.neural_network import MLPRegressor
import xgboost as xgb

# Each model will have 3 hyperparameter settings
model_params = {
    "Random Forest": [
        {"n_estimators":50, "max_depth":5},
        {"n_estimators":100, "max_depth":10},
        {"n_estimators":150, "max_depth":15}
    ],
    "LSVM": [
        {"C":0.5, "epsilon":0.2},
        {"C":1.0, "epsilon":0.1},
        {"C":2.0, "epsilon":0.05}
    ],
    "XGBoost": [
        {"n_estimators":50, "max_depth":3, "learning_rate":0.1},
        {"n_estimators":100, "max_depth":6, "learning_rate":0.1},
        {"n_estimators":150, "max_depth":9, "learning_rate":0.05}
    ],
    "KNN": [
        {"n_neighbors":3, "weights":"uniform"},
        {"n_neighbors":5, "weights":"distance"},
        {"n_neighbors":7, "weights":"uniform"}
    ],
    "Neural Network": [
        {"hidden_layer_sizes":(25,25), "activation":"relu", "max_iter":1000},
        {"hidden_layer_sizes":(50,50), "activation":"relu", "max_iter":1000},
        {"hidden_layer_sizes":(100,50), "activation":"tanh", "max_iter":1000}
    ]
}

print("Models and hyperparameters defined")
```

```

results = []

for model_name, param_list in model_params.items():
    for params in param_list:
        # Instantiate model
        if model_name == "Random Forest":
            model = RandomForestRegressor(random_state=42, **params)
        elif model_name == "LSVM":
            model = SVR(**params)
        elif model_name == "XGBoost":
            model = xgb.XGBRegressor(random_state=42, **params)
        elif model_name == "KNN":
            model = KNeighborsRegressor(**params)
        elif model_name == "Neural Network":
            model = MLPRegressor(random_state=42, **params)

        # Decide which data to use (scaled or original)
        if model_name in scaled_models:
            model.fit(X_train_scaled, y_train)
            y_train_pred = model.predict(X_train_scaled)
            y_test_pred = model.predict(X_test_scaled)
        else:
            model.fit(X_train, y_train)
            y_train_pred = model.predict(X_train)
            y_test_pred = model.predict(X_test)

        # Metrics
        train_mse = mean_squared_error(y_train, y_train_pred)
        test_mse = mean_squared_error(y_test, y_test_pred)
        train_rmse = np.sqrt(train_mse)
        test_rmse = np.sqrt(test_mse)
        train_mae = mean_absolute_error(y_train, y_train_pred)
        test_mae = mean_absolute_error(y_test, y_test_pred)
        train_r2 = r2_score(y_train, y_train_pred)
        test_r2 = r2_score(y_test, y_test_pred)

        results.append({
            "Model": model_name,
            "Hyperparameters": str(params),
            "Training MSE": round(train_mse, 2),
            "Testing MSE": round(test_mse, 2),
            "Training RMSE": round(train_rmse, 2),
            "Testing RMSE": round(test_rmse, 2),
            "Training MAE": round(train_mae, 2),
            "Testing MAE": round(test_mae, 2),
            "Training R²": round(train_r2, 2),
            "Testing R²": round(test_r2, 2)
        })

results_df = pd.DataFrame(results)
results_df.sort_values(by="Testing RMSE", inplace=True)
results_df.reset_index(drop=True, inplace=True)
results_df

```

Chart

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# -----
# 1. Prepare the results data (best hyperparams per model)
# -----
best_models = pd.DataFrame({
    'Model': ['Random Forest', 'XGBoost', 'Neural Network', 'KNN', 'LSVM'],
    'Training RMSE': [1109.74, 896.41, 2276.47, 2904.56, 8905.13],
    'Testing RMSE': [1706.97, 1872.05, 1995.03, 3541.66, 8749.46],
    'Training R²': [0.98, 0.99, 0.94, 0.90, 0.02],
    'Testing R²': [0.96, 0.96, 0.95, 0.84, 0.02]
})

# Baseline model values
baseline_rmse = 2156.21
baseline_r2 = 0.94

x = np.arange(len(best_models))
width = 0.35

# -----
# 2. Plot RMSE with baseline
# -----
fig, ax = plt.subplots(figsize=(10,6))
rects1 = ax.bar(x - width/2, best_models['Training RMSE'], width, label='Training RMSE', color='skyblue')
rects2 = ax.bar(x + width/2, best_models['Testing RMSE'], width, label='Testing RMSE', color='salmon')

# Horizontal line for baseline
ax.axhline(y=baseline_rmse, color='black', linestyle='--', label='Baseline RMSE (Linear Regression)')

ax.set_ylabel('RMSE (ε)')
ax.set_title('Model Comparison: Training vs Testing RMSE with Baseline')
ax.set_xticks(x)
ax.set_xticklabels(best_models['Model'], rotation=45, ha='right')
ax.legend()
plt.tight_layout()
plt.show()

# -----
# 3. Plot R² with baseline
# -----
fig, ax = plt.subplots(figsize=(10,6))
rects1 = ax.bar(x - width/2, best_models['Training R²'], width, label='Training R²', color='lightgreen')
rects2 = ax.bar(x + width/2, best_models['Testing R²'], width, label='Testing R²', color='orange')

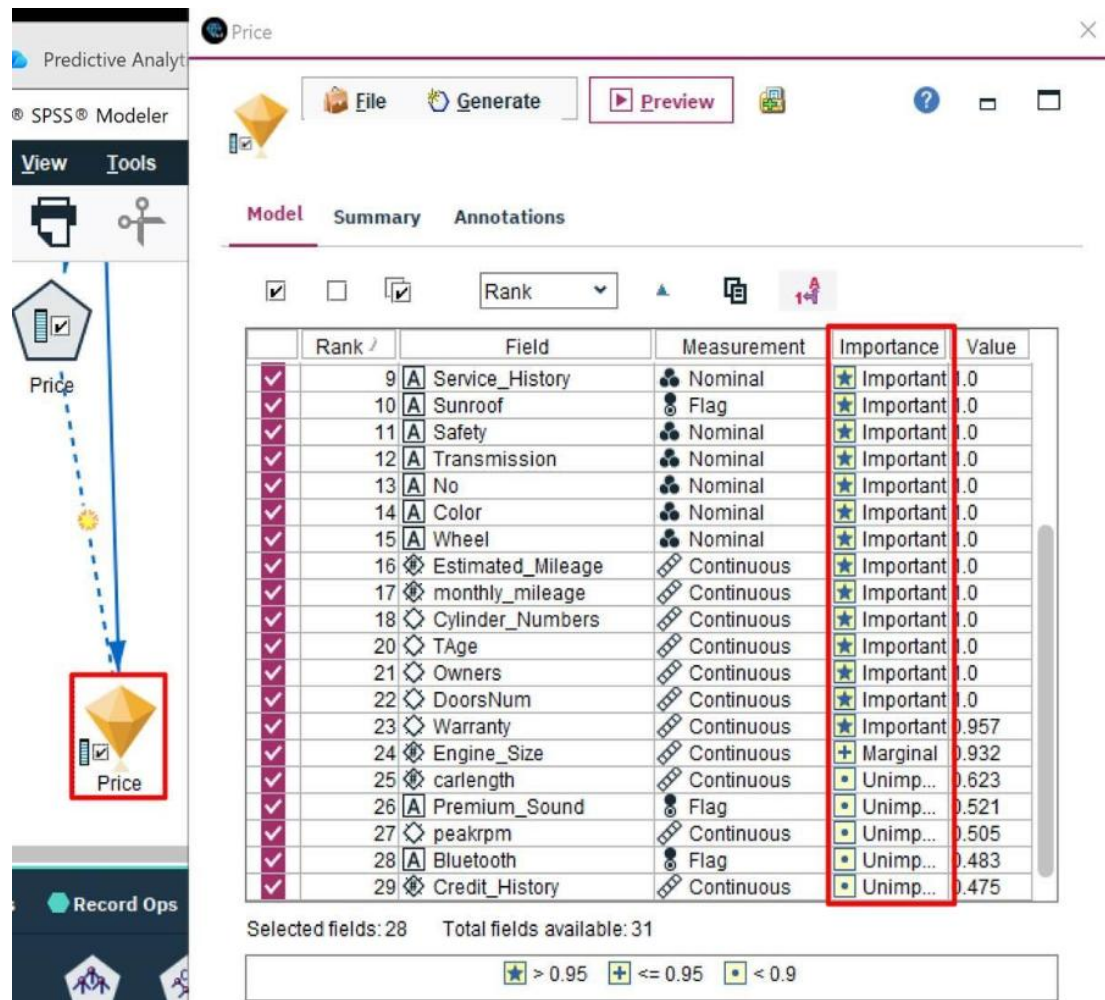
# Horizontal line for baseline
ax.axhline(y=baseline_r2, color='black', linestyle='--', label='Baseline R² (Linear Regression)')

ax.set_ylabel('R²')
ax.set_title('Model Comparison: Training vs Testing R² with Baseline')
ax.set_xticks(x)
ax.set_xticklabels(best_models['Model'], rotation=45, ha='right')
ax.legend()
plt.tight_layout()
plt.show()
```

IBM SPSS Modeler

Variable Importance Rank

Feature Selection Node



Predictive Analyt

SPSS® Modeler

View Tools

Price

Model Summary Annotations

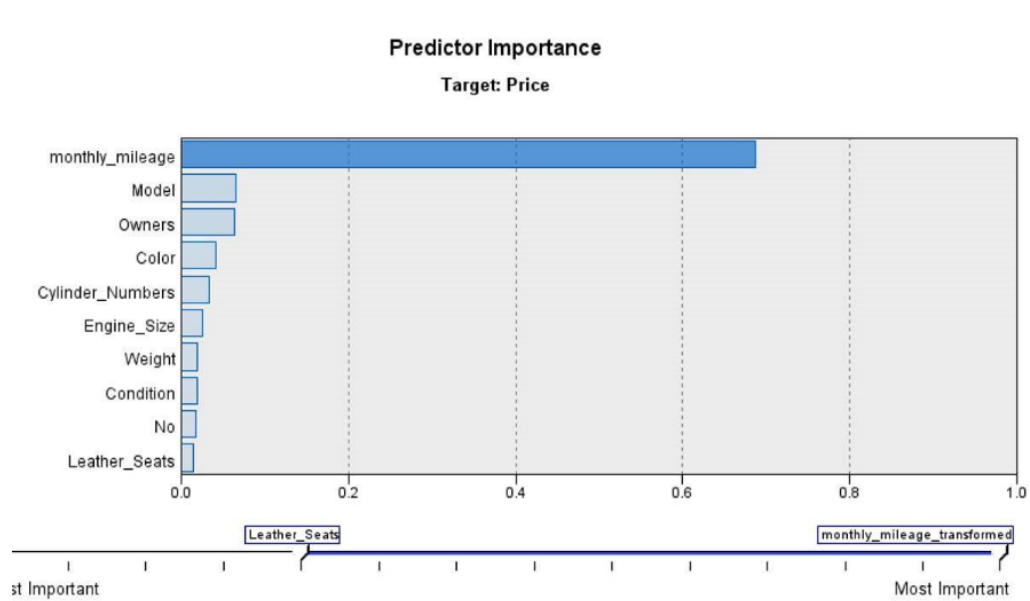
Rank

Rank	Field	Measurement	Importance	Value
9	Service_History	Nominal	Important	1.0
10	Sunroof	Flag	Important	1.0
11	Safety	Nominal	Important	1.0
12	Transmission	Nominal	Important	1.0
13	No	Nominal	Important	1.0
14	Color	Nominal	Important	1.0
15	Wheel	Nominal	Important	1.0
16	Estimated_Mileage	Continuous	Important	1.0
17	monthly_mileage	Continuous	Important	1.0
18	Cylinder_Numbers	Continuous	Important	1.0
20	TAge	Continuous	Important	1.0
21	Owners	Continuous	Important	1.0
22	DoorsNum	Continuous	Important	1.0
23	Warranty	Continuous	Important	0.957
24	Engine_Size	Continuous	Marginal	0.932
25	carlength	Continuous	Unimp...	0.623
26	Premium_Sound	Flag	Unimp...	0.521
27	peakrpm	Continuous	Unimp...	0.505
28	Bluetooth	Flag	Unimp...	0.483
29	Credit_History	Continuous	Unimp...	0.475

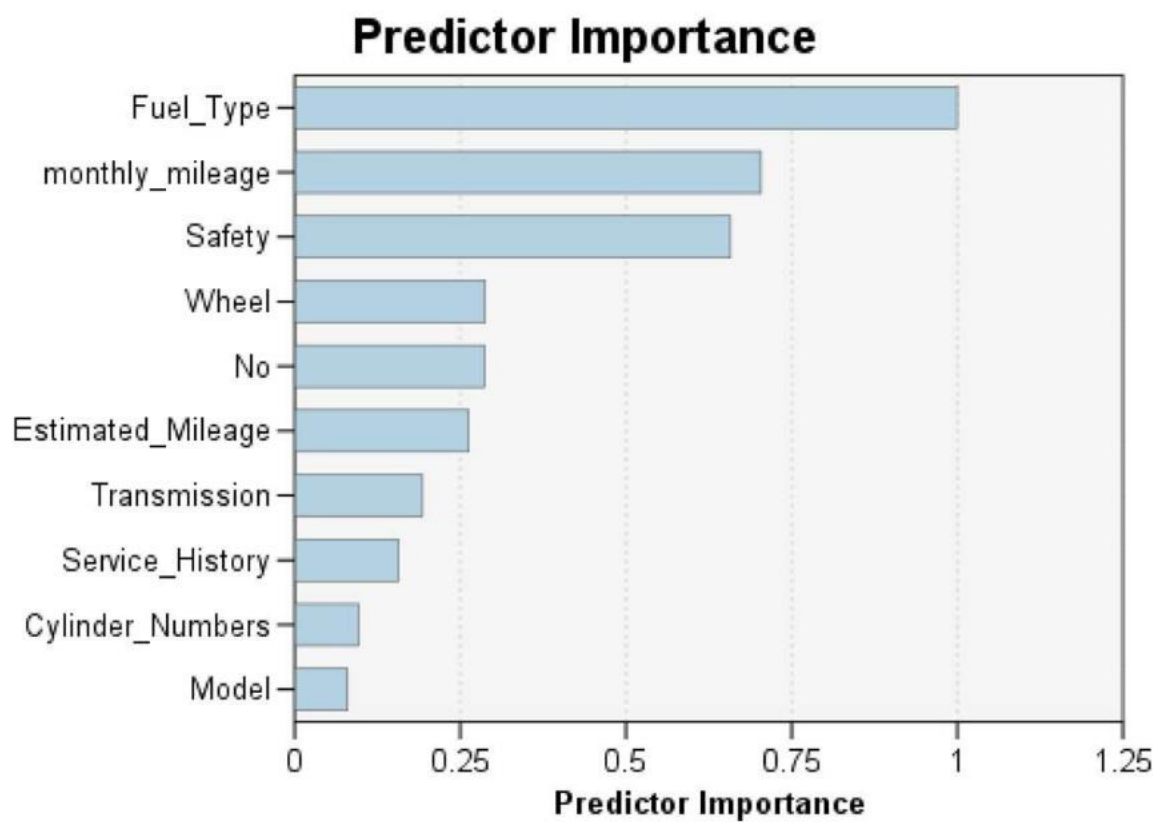
Selected fields: 28 Total fields available: 31

> 0.95 <= 0.95 < 0.9

Regression Model



Random Forest



The top 10 inputs are shown